

1Voucher — How the system works

From onboarding to value movement

A Web2-first fintech with an onchain compliance and settlement layer. Consumers see a simple balance; the protocol holds claims in a Vault backed by treasury tokens (e.g. TTZA, TTMW).

Consumer app

- Passkey wallet
- Spend / save balances
- Pay, remit, redeem vouchers

Admin console

- Countries & currencies
- Treasury ops
- Merchants & compliance

Merchant console

- POS & products
- Accept payments
- Settle to fiat

Phase 1 pilot: ZA corridor live; additional corridors (e.g. MWK / TTMW) follow the same pattern.

Three setup paths before money moves

Actor	What they configure	What it unlocks
Admin	Countries, fiat currencies, treasury tokens, corridors, KYC tiers	Which markets exist and which onchain tokens back them
Merchant	Business profile, wallet, settlement preference (fiat or onchain)	Ability to receive consumer payments
Consumer	Passkey, country, display name, optional ENS tag	Personal Safe wallet + Vault balance claim

Admin: countries, currencies & treasury tokens

1 Add country

Countries table links ISO code → fiat currency (e.g. ZA → ZAR, MW → MWK). Controls KYC tiers and product availability.

2 Add treasury currency row

Currencies page: type TREASURY, code TTZA/TTMW, fiat anchor (ZAR/MWK). Saves to DB only — no chain tx yet.

3 Deploy corridor token (explicit action)

Deploy button creates ERC-1967 proxy + initialize; registers stablecoins row; wires `Vault.setCurrencyTreasuryToken(fiat, proxy)`.

4 Configure corridors & partners

Remittance corridors, payout partners, FX overrides — off-chain routing for cross-border exits.

Saving a currency never auto-deploys onchain. Deploy requires admin confirmation and deployer wallet gas.

Consumer registration → live wallet

- 1 **Passkey created on device** — WebAuthn P-256 key, no seed phrase.
- 2 **Backend resolves Safe signer** — SafeWebAuthnSignerFactory → deterministic signer address.
- 3 **Consumer.registerConsumer onchain** — Deploys Safe proxy; stores country hash, name hash, KYC level 0.
- 4 **Off-chain profile (best-effort)** — idOS credential, ENS subdomain, Pimlico paymaster whitelist.
- 5 **consumers row in Postgres** — Links wallet, country, KYC, registration step for retries.

Result: consumer has a gasless-capable Safe address registered onchain. Balance is still zero until cash-in.

Merchant registration → can receive payments

- 1 **Wallet signs nonce** — POST /api/merchants/register proves wallet ownership.
- 2 **merchants row** — Name, country, settlement type, verification PENDING.
- 3 **Onchain whitelist (best-effort)** — TreasuryToken.addToWhitelist;
Vault.setTrustedCounterparty.
- 4 **Admin verification** — Admin approves KYB → merchant goes live.

Trusted merchants bypass consumer-to-consumer country matching — consumers can pay them without P2P restrictions.

What consumers actually hold

Consumers see one fungible balance (e.g. R500). They never hold TTZA/TTMW ERC-20 in their wallet.

Spend balance (ZAR claim)

- Vault ledger entry — ERC-4626-style shares pinned 1:1
- Backed by TTZA sitting in the Vault
- Flat 1:1 — no yield to consumer

Treasury token (TTZA, TTMW...)

- Protocol bank-cash token — permissioned ERC-20
- Mint on fiat-in; swept on merchant settlement
- Closed-loop, domestic-only transfers

Accounting: TT in vault $\approx$ sum of all ZAR claims + unallocated pool. USD save balance is separate (USDC reserve).

Cash-in: how value enters the system

After registration, a consumer needs a funded Vault claim before they can pay anyone.

- 1 **Off-chain fiat or voucher** — Bank deposit reference or voucher number in `deposit_references`.
- 2 **Mint treasury token → Vault** — Backend mints TTZA/TTMW to Vault contract.
- 3 **Credit consumer claim** — `Vault.adminCredit(wallet, amount, ZAR)`.
- 4 **Audit trail** — `deposit_references` + indexer chain events.

POC: Admin Treasury → dev-credit simulates this on Sepolia. Production gates mint on reconciled bank deposit.

After cash-in: how value moves

Flow	Mechanism	Onchain effect
Pay merchant	Consumer signs Vault.transfer → backend relays	Ledger debit/credit; merchant accumulates ZAR claim
P2P transfer	Passkey-signed Vault.transfer (same country)	Claim moves wallet → wallet
WhatsApp escrow	Transfer to custodial escrow + claim link	Held until recipient onboarded
ZAR ↔ USD convert	adminDebit + adminCredit at FX – spread	Reallocates claims; may mint TT if pool short
Remittance	Lock → partner payout → settle	Burn/sweep TT, debit claim
Merchant settlement	Admin approves → debit merchant claim	TT swept to platform treasury

Onchain architecture

Consumer.sol

- Safe wallet factory
- Identity + KYC registry
- Compliance event log

Vault.sol

- Unified balance ledger
- Remittance + FX paths
- ERC-4626 reserve semantics

TreasuryToken.sol

- Shared logic (one impl)
- Compliance in _update hook
- Mint/burn/freeze/whitelist

Each corridor (TTZA, TTMW...) is a separate **ERC-1967 proxy instance** — same logic, different `initialize(name, symbol)`.

Pattern	Detail
Upgrade model	UUPS proxies — stable addresses, swappable implementations
Consumer wallets	Safe v1.4.1 + WebAuthn signer; ERC-4337 paymaster planned
Two platform wallets	Deployer (cold admin) vs Backend (hot ops + gas)

Database (PostgreSQL)

System config

- countries, currencies, stablecoins
- kyc_levels, corridors, payout_partners
- contract_deployments

Identity & commerce

- consumers, webauthn_credentials
- merchants, products, merchant_sales
- settlement_requests

Money movement audit

- deposit_references
- consumer_conversions
- change_vouchers, pending_claims

Chain mirror

- chain_events + indexer_cursor
- protocol_gas_costs
- app_config

API layer (Node.js REST)

/api — public & authenticated

- /register, /auth/passkey/*
- /consumer/* — balance, convert, transfer
- /merchants/* — signup, POS
- /countries, /fx/rate

/api/admin — operator

- Treasury dev-credit, harvest
- CRUD countries, currencies
- Reports: revenue, events
- treasury-instances

Backend orchestrates multi-step flows; contracts enforce compliance gates. JWT roles control UI access — separate from onchain AccessControl.

Summary

End-to-end: setup → funded → payment

- 1 **Admin enables ZA + TTZA** — Country, currency, deploy proxy, wire Vault.
- 2 **Merchant registers + admin verifies** — Whitelisted on TT + Vault trusted.
- 3 **Consumer registers** — Safe deployed, KYC level 0, zero balance.
- 4 **Cash-in** — Fiat/voucher → mint TT to Vault → credit ZAR claim.
- 5 **Consumer pays merchant** — Signed Vault.transfer; merchant ZAR claim increases.
- 6 **Merchant settles (optional)** — Claim debited, TT swept; bank payout off-chain.

Smart contracts = compliance ledger + settlement. Backend = orchestration, KYC, partners, and UX.

Source: 1Voucher codebase & admin docs · Sepolia pilot · July 2026